Guida introduttiva

Il framework per gli agenti.

Ogni pagina è un componente React su un canvas 1920 × 1080.

Premi → per iniziare

```
$ bun x @open-slide/cli init
```

L'idea

Una slide è un file.

Nessun DSL, nessun linguaggio di template. Ogni pagina è un componente React, e il default export è il deck.

slides/hello/index.tsx — TSX · 14 LINES

```tsx
import type { Page } from '@open-slide/core';

const WORD = 'deck';

const Cover: Page = () => (
  <div style={{ padding: 160, background: '#fff' }}>
    <h1 style={{ fontSize: 188, letterSpacing: '-0.04em' }}>
      Hello, <em style={{ color: '#de3b3d' }}>{WORD}</em>.
    </h1>
    <p>Una slide React, renderizzata dal vivo.</p>
  </div>
);

export default [Cover] satisfies Page[];
```

rendered output — LIVE



01 · Avvio

Un solo comando bun.

Bun esegue lo scaffolder dall'inizio alla fine: dipendenze installate, git inizializzato, dev server a un comando. Nessuna installazione globale, nessuna config da toccare.

~/code — zsh

```
$ bun x @open-slide/cli init my-deck

✓ Created open-slide workspace in ~/code/my-deck
✓ Installed dependencies with bun
✓ Initialized git repository with first commit

Next steps:
  cd my-deck
  bun run dev

Then open the dev server and start authoring in slides/<your-slide>/.
$
```

02 · Prompt

Chiedi al tuo agente.

Un semplice prompt /create-slide abbozza ogni pagina come un vero componente.

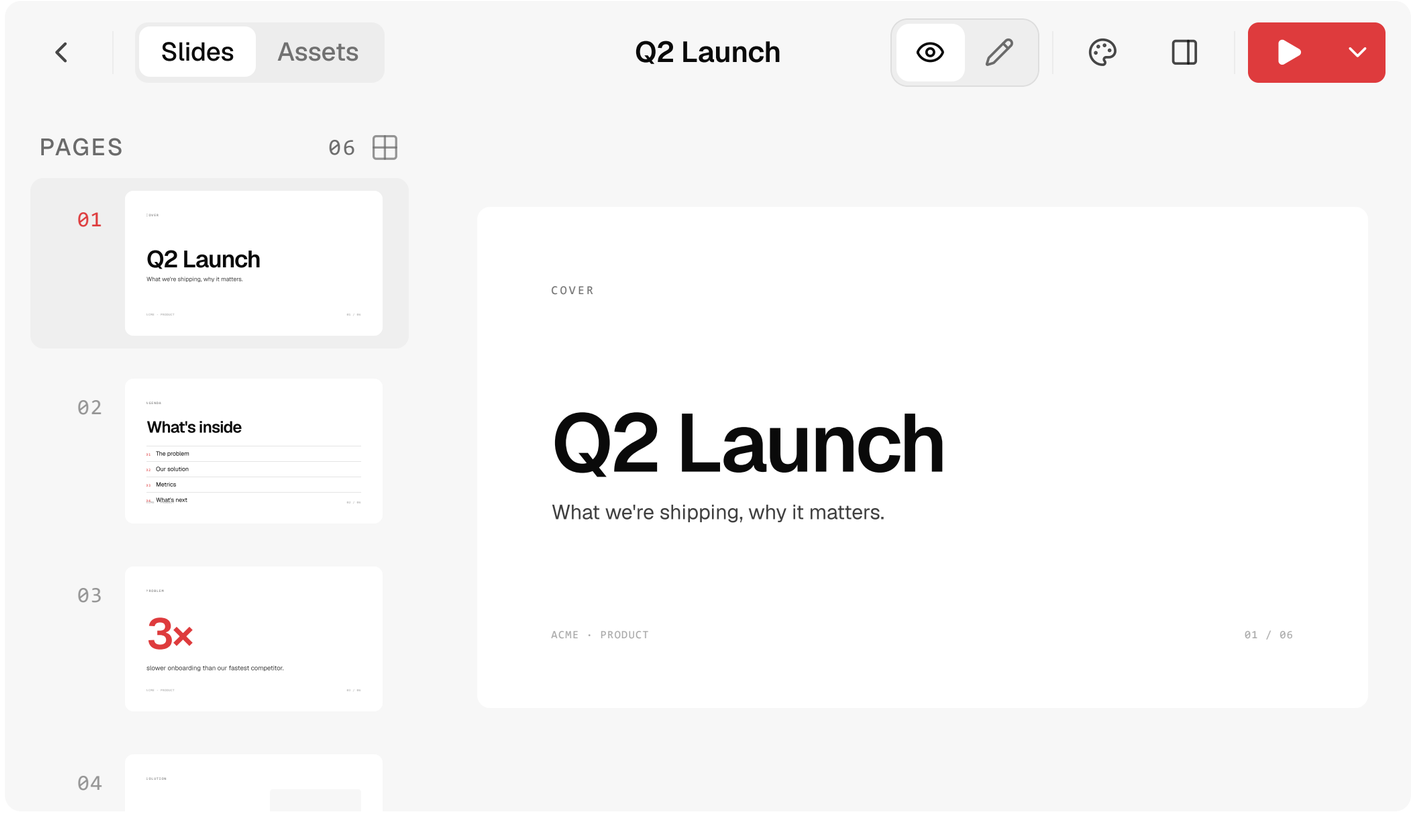
claude · ~/my-deck

```
› /create-slide the Q2 launch deck

Drafting 6 pages…
write slides/q2-launch/index.tsx
hmr localhost:5173/s/q2-launch ✓
Done. Cover, Agenda, Problem,
Solution, Metrics, Next.
›
```

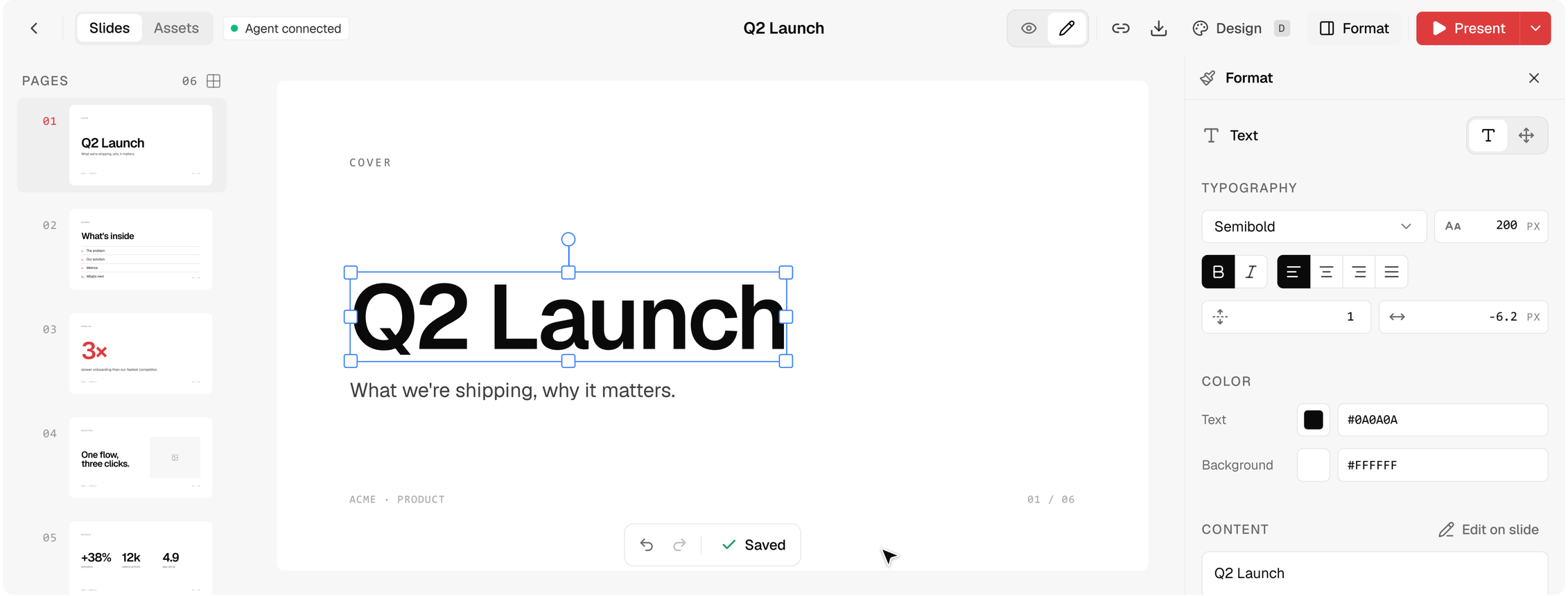
Claude Code · Codex · Cursor · Gemini …

ANY AGENT



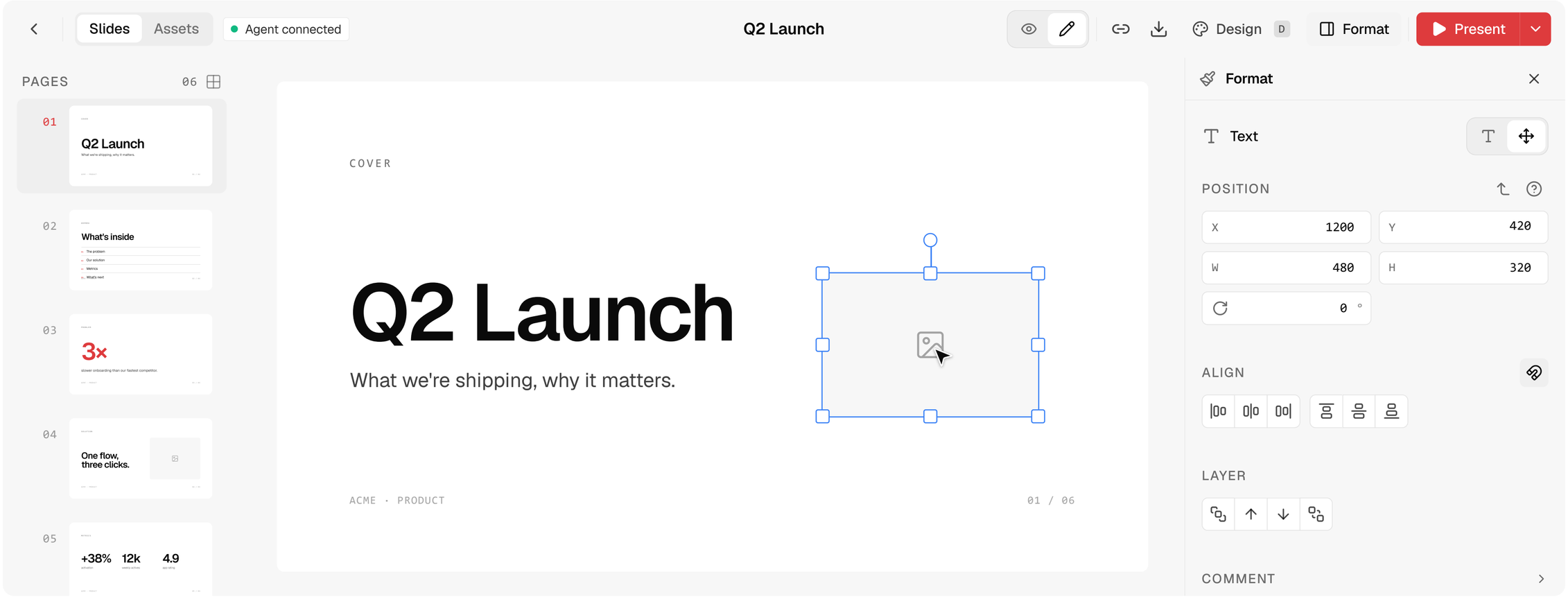
03 · Modifica

Clicca un elemento. Ritoccalo. Salva una volta.



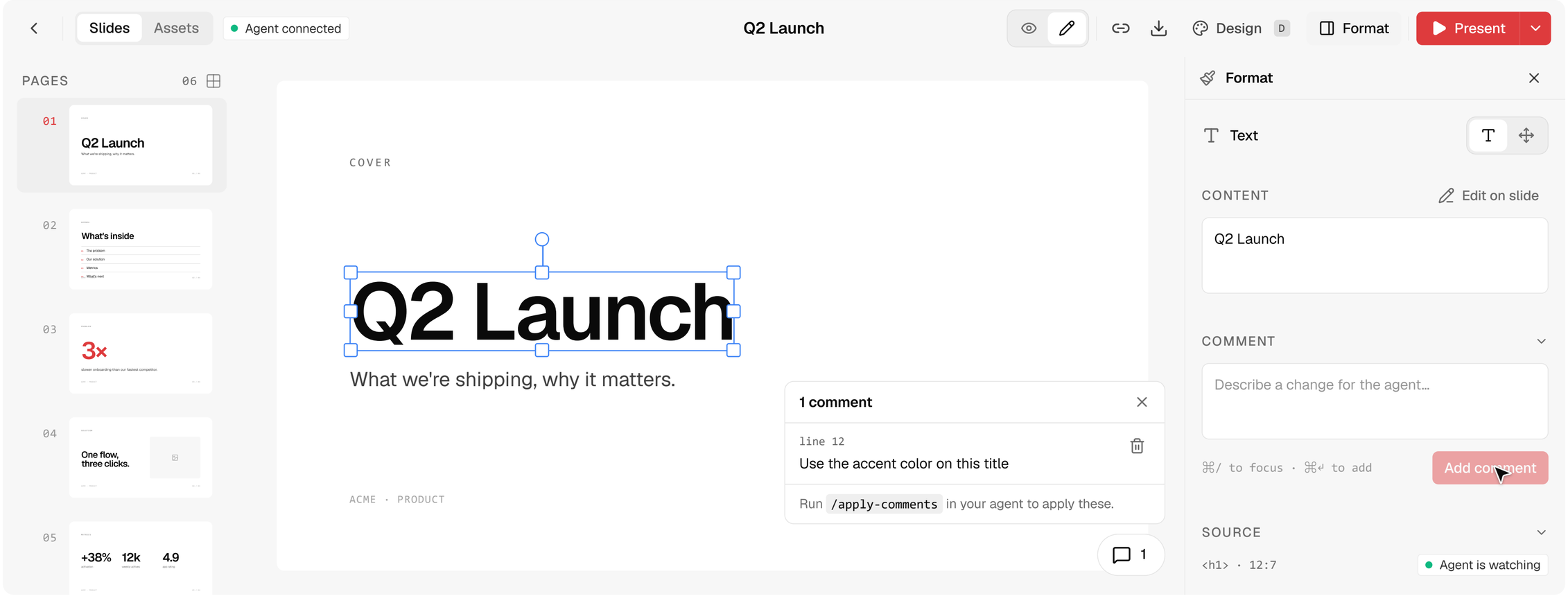
03 · Modifica

Trascina. Allinea. Aggancia.



04 · Commento

Indica cosa non va.



04 · Applica

L'agente lo applica.

Esegui /apply-comments. L'agente legge ogni marker, modifica esattamente ciò che hai segnalato e lo rimuove.

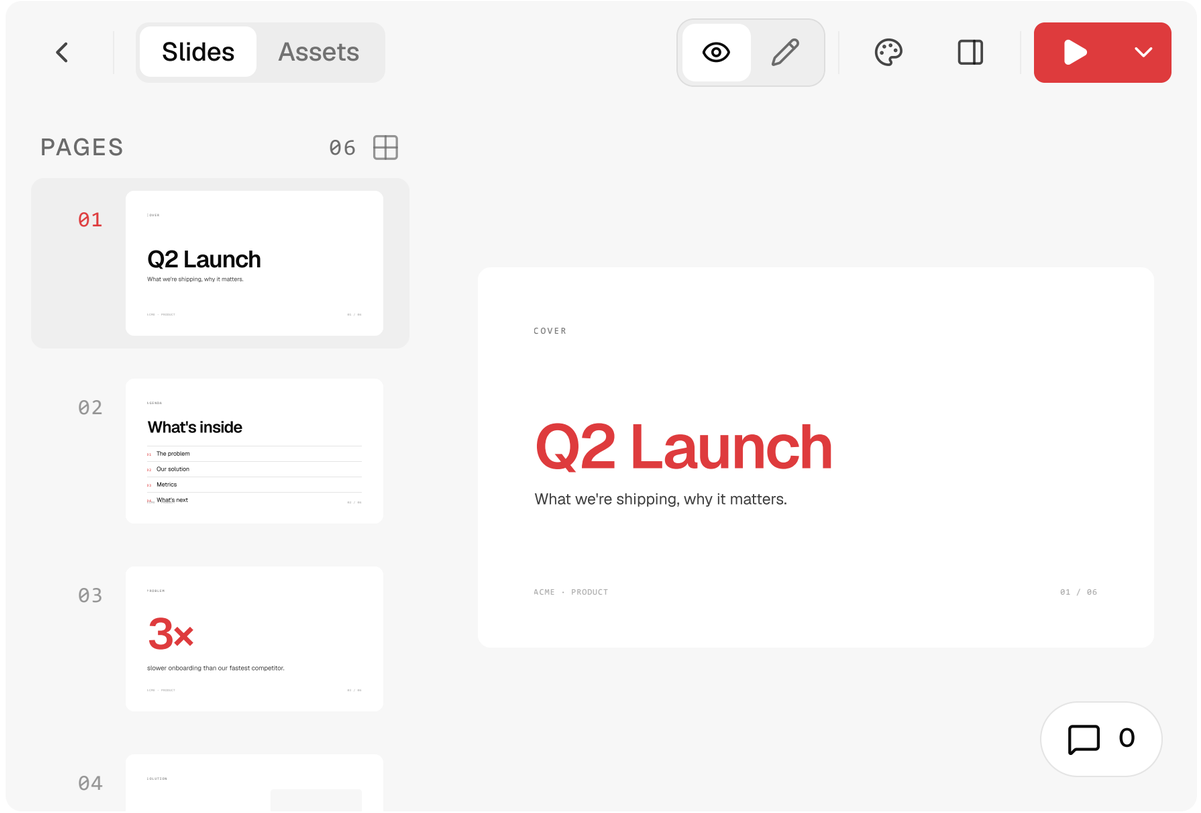
claude · ~/my-deck

```
› /apply-comments

1 comment found. Applying…

11  <section>
12  {/* @slide-comment id="c-a1b2c3d4" text="Use the accent co…
13    <h1 style={{ color: '#de3b3d' }}>Q2 Launch</h1>
14  </section>

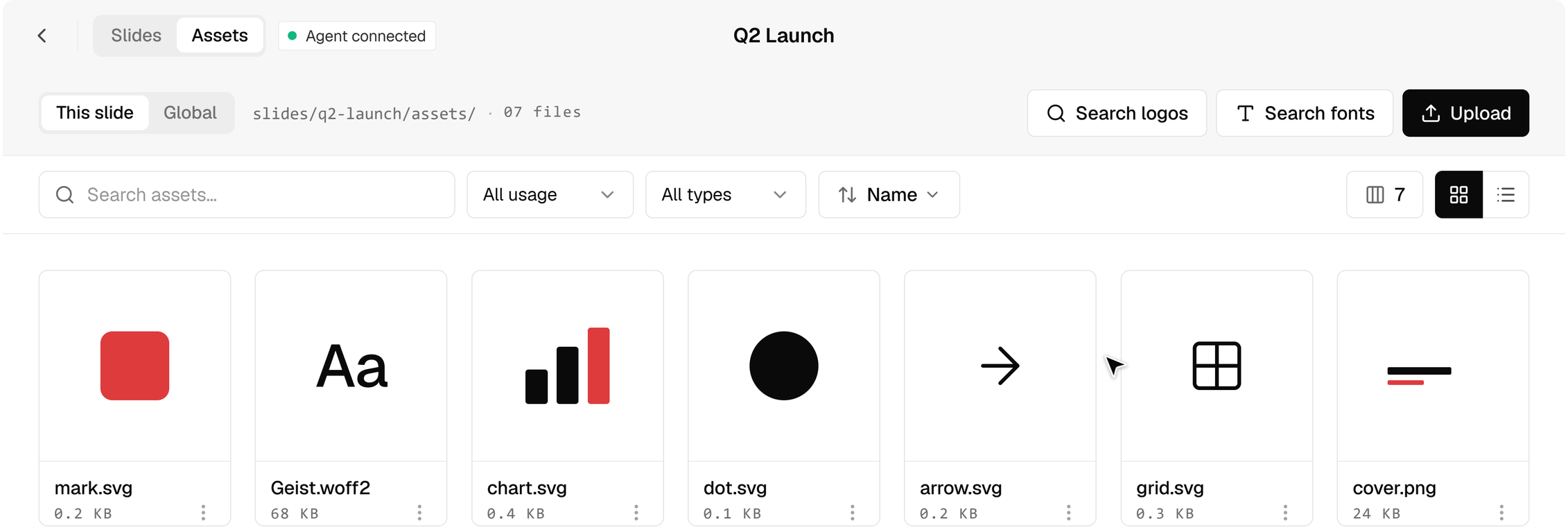
edit slides/q2-launch/index.tsx
✓ 1 comment applied
```



Strumenti

Gli asset, tutti in un posto.

Trascina file, cerca loghi su svgl, prendi Google Fonts. Tutto finisce in slides/<id>/assets/, pronto da importare.



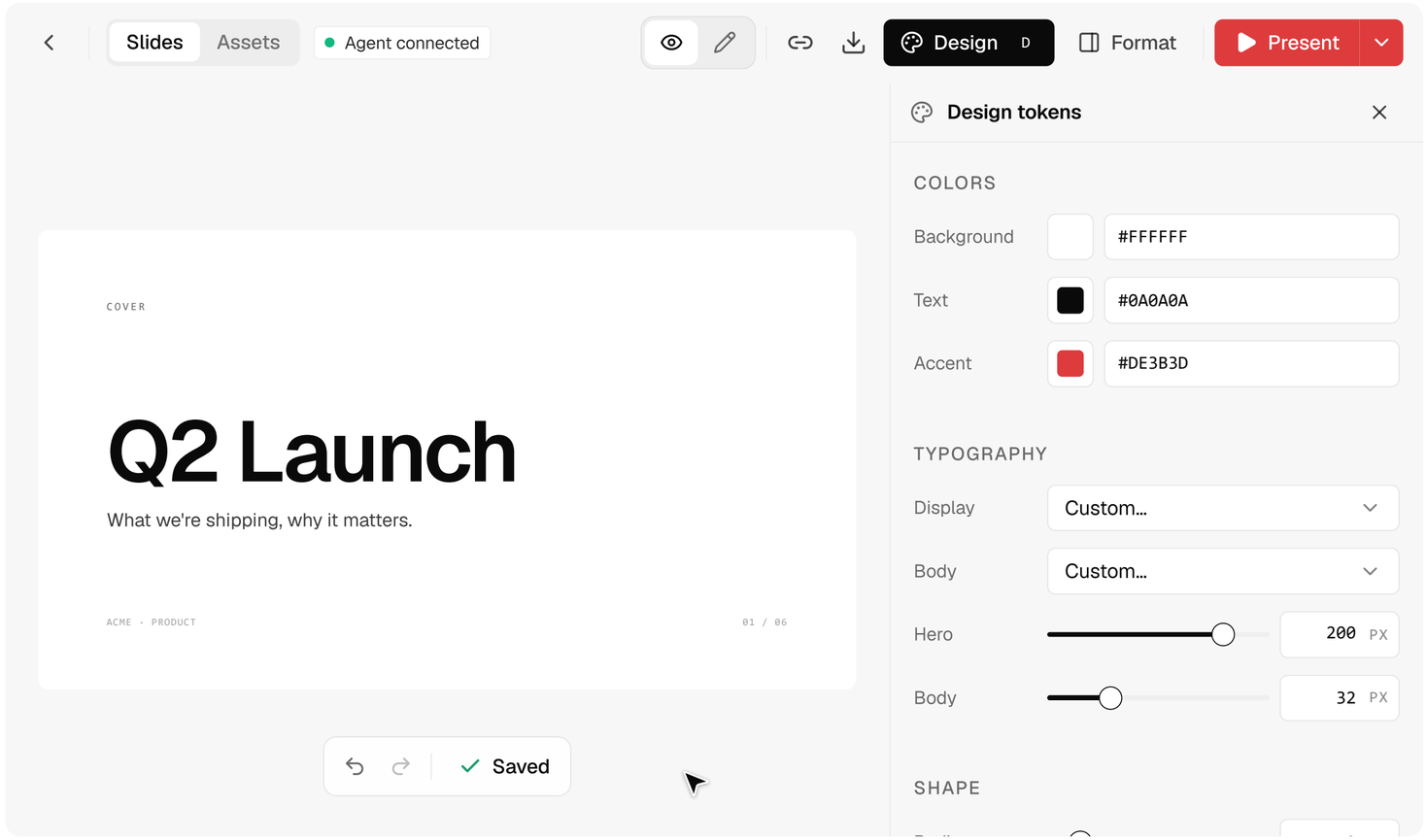
Strumenti

I design token, dal vivo.

slides/q2-launch/index.tsx DESIGN

```ts
export const design: DesignSystem = {
  palette: {
    bg: '#ffffff',
    text: '#0a0a0a',
    accent: '#de3b3d',
  },
  fonts: {
    display: 'Geist, sans-serif',
    body: 'Geist, sans-serif',
  },
  typeScale: { hero: 200, body: 32 },
  radius: 12,
};

// read as var(--osd-size-hero)
```



Strumenti

Fissa un look. Riusalo.

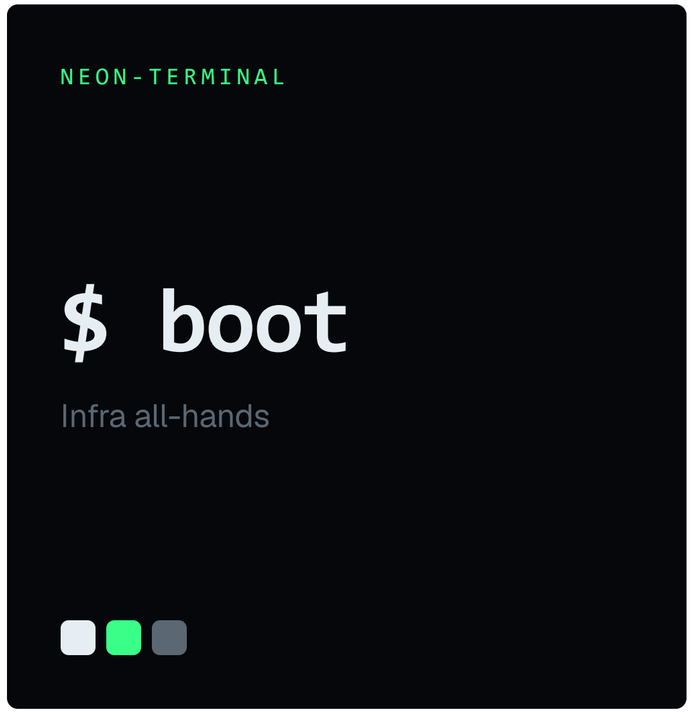
/create-theme scrive themes/<id>.md: palette, tipografia e componenti Title e Footer pronti da incollare.
/create-slide lo legge prima di scrivere.



themes/editorial-noir.md — DARK · SERIF



themes/paper-press.md — LIGHT · SERIF

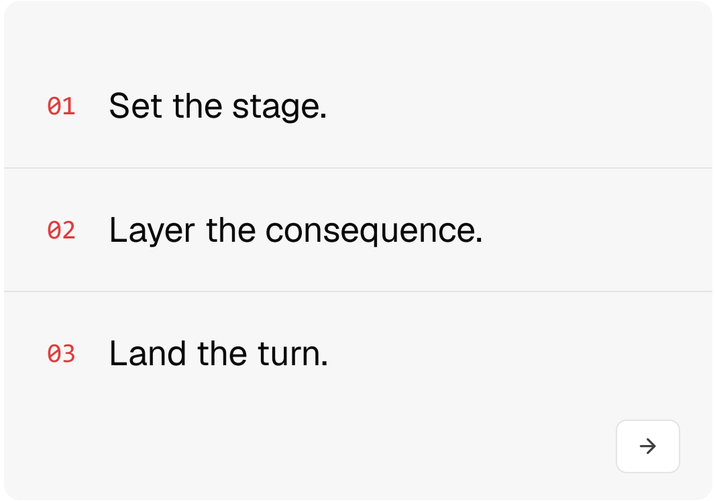


themes/neon-terminal.md — DARK · MONO

Strumenti

Il movimento, in tre primitive.

Rivelazioni a step, una transizione di casa e morph tra elementi condivisi. Tutto dichiarato nel file.



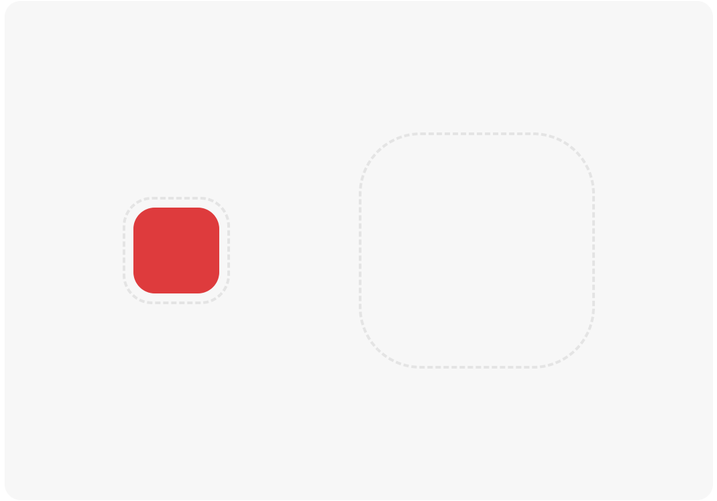
`<Steps> / <Step>`

Ogni → rivela il passo successivo. Entrando da fuori la pagina è già completa.



`SlideTransition`

Un solo DNA per deck: salita di 6 px, 200 ms, ease-out. Questo deck la usa.



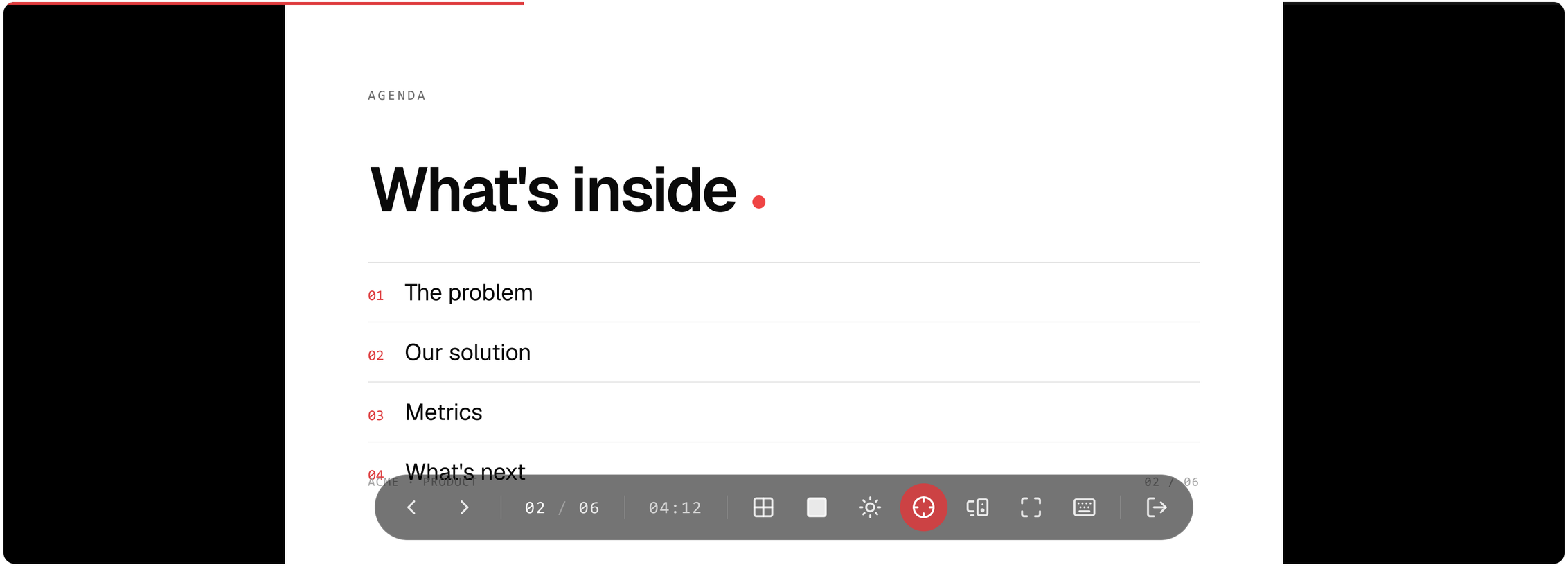
`MorphElement`

Stesso id su due pagine: l'oggetto scivola dall'una all'altra. Il quadratino dell'eyebrow l'ha appena fatto.

05 · Presenta

Presenta dal browser.

F per il fullscreen, Enter per una finestra. Panoramica, laser, blackout e mappa tastiera sono a un tasto di distanza.



05 · Presenta

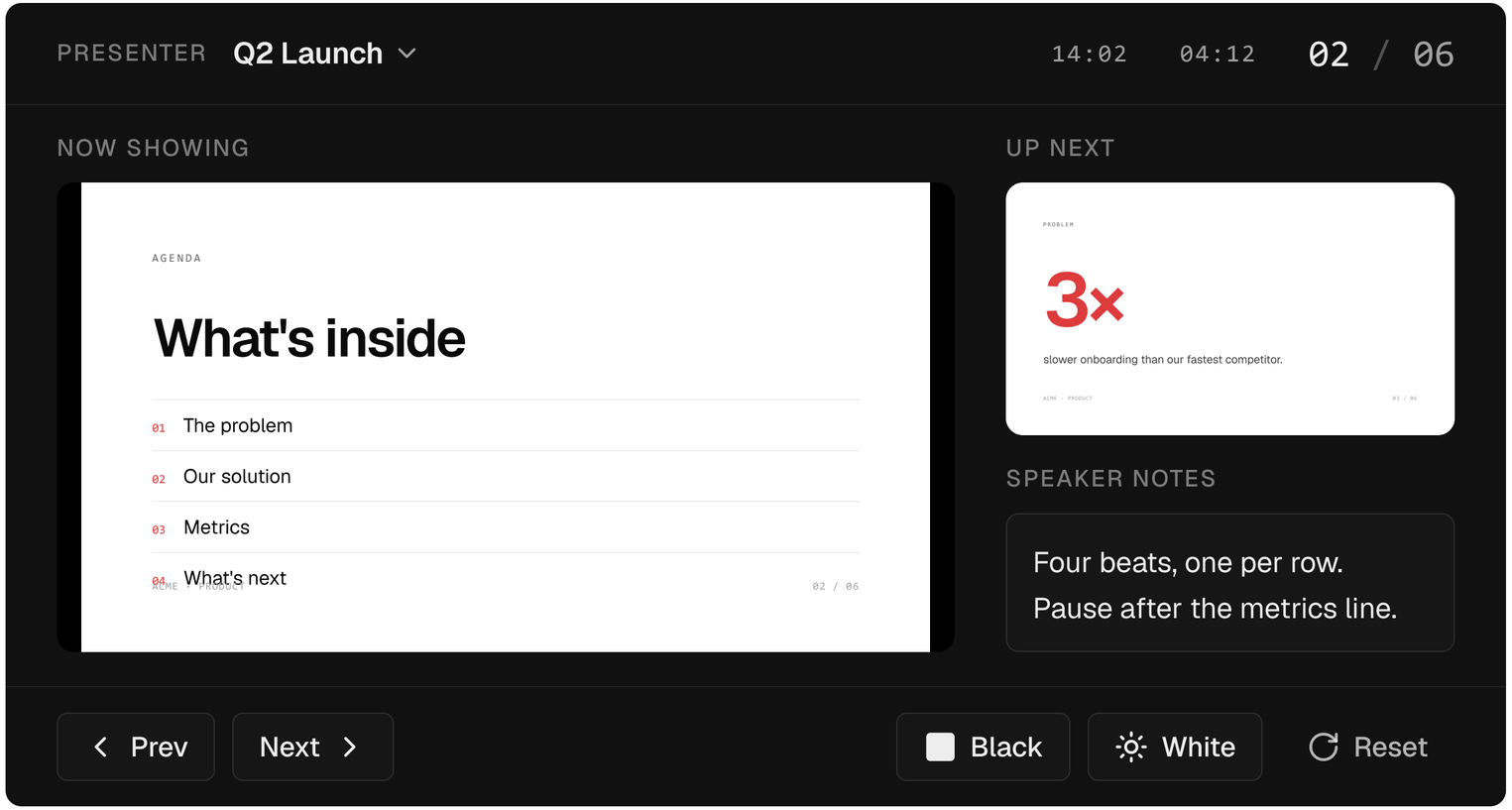
Le note viaggiano con il deck.

Le note vivono accanto alle pagine. La presenter view mostra la slide corrente, la prossima e il tuo copione, con un timer.

slides/q2-launch/index.tsx NOTES

```
export const notes = [
  'Open with the customer quote.',
  'Four beats, one per row.',
  'Let the 3× land. Then breathe.',
  undefined,
  'Cite the Q1 baseline first.',
  'End on the waitlist link.',
];

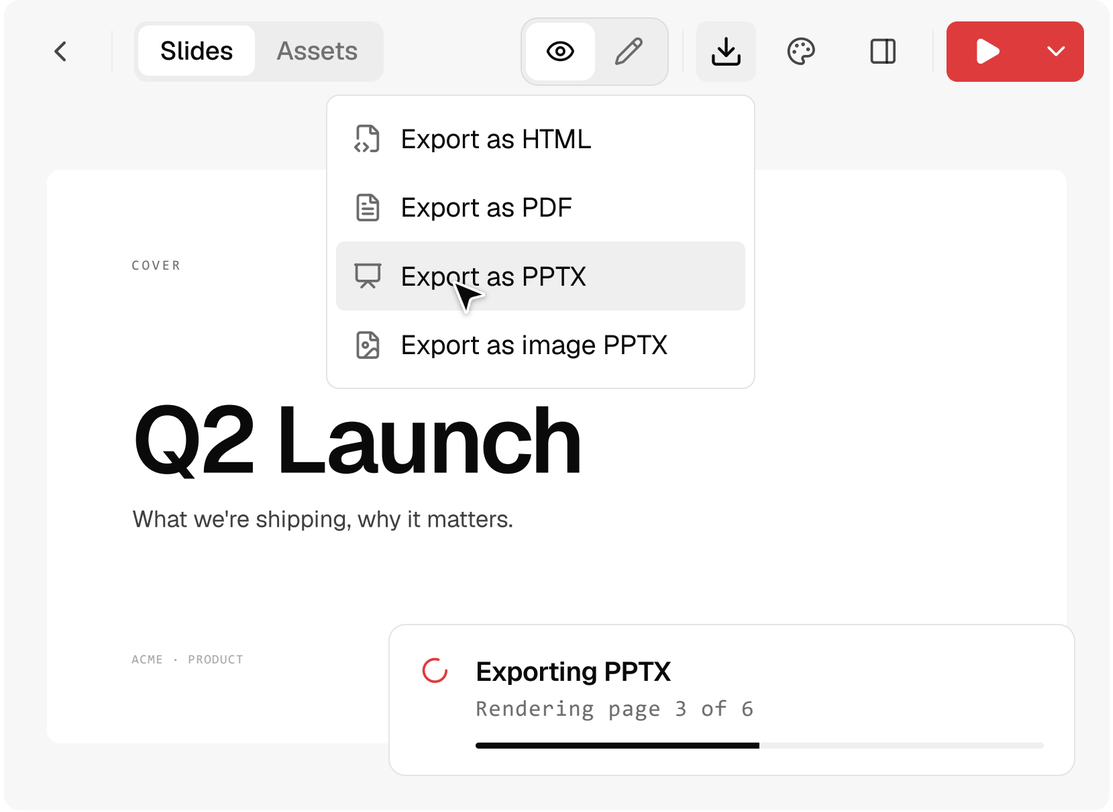
// index-aligned with the pages
```



Pubblica

Esportalo. O pubblicalo.

Scarica HTML, PDF o un PPTX modificabile dall'editor. Oppure fai il build e carica dist/ su qualsiasi host statico.



~/my-deck — zsh

```
$ bun run build
vite v7 building for production…
✓ built in 1.2s → dist/

$ bun run preview
➜  Local: http://localhost:4173/
$
```

Perché open-slide

Porta il tuo agente.

Ogni workspace arriva con regole e skill per gli agenti. Le slide sono .tsx puri: niente SDK, niente lock-in.

~/my-deck

```
AGENTS.md  house rules
CLAUDE.md
.agents/skills/
├─ create-slide
├─ slide-authoring
├─ apply-comments
└─ create-theme
.claude/skills/  → .agents/skills
slides/
themes/
```

AGENTS.md

Regole che ogni agente legge.

Skill integrate

Scrivere, modificare, temare, applicare commenti.

React puro

Se sa modificare codice, sa modificare slide.

Perché open-slide

Tuo. Per sempre.

Ogni slide è un file nel tuo repo. Fai diff, review, branch. Nessun database proprietario, nessun esporta-e-spera.

~/my-deck — git log

```
$ git log --oneline slides/q2-launch/

* a1b2c3d (HEAD -> main) refine cover typography
* 9f8e7d6 revise Q2 metrics after finance review
* 6a5b4c3 apply comments from design crit
* 3c2d1e0 initial draft of the Q2 launch deck

$
```

File .tsx puri

Nessun formato proprietario.

Con diff in qualsiasi tool

Rivedile come una PR.

Branch · merge · revert

Gli strumenti che già conosci.

Riepilogo

Il ciclo è tutto qui.

01

Avvio

`bun x @open-slide/cli init`

02

Prompt

`/create-slide`

03

Modifica

`clic · ritocca · salva`

04

Commento

`/apply-comments`

05

Presenta

`F · schermo intero`

Modifica `slides/<your-slide>/index.tsx` e guardalo aggiornarsi dal vivo.